实验 3 基于流式套接字的文件传输程序

实验目的

1、掌握流式套接字编程框架。

2、会接收长度可变的应用层报文。

3、掌握使用流式套接字传输文件的技术，会使用断点重传技术。

实验内容

分别设计和实现服务器端和客户端网络通信程序，使得在这两者之间能够传输文件。要求有：

1、流式套接字编程基本框架。其中服务器采用循环方式工作，即每次只为一个客户提供服务，对客户提供的服务采用串行方式进行。

2、掌握可变长度报文的发送和接收。

3、客户端能够指定从任意文件的任意偏移量处开始下载文件。

4、使用流式套接字传输文件。

实验指导

任务一：文件传输程序设计

1、服务器和客户端程序的工作流程可设计如下：

（1）服务器端软件首先启动，在双方约定的端口上进行监听；

（2）启动客户端程序，输入要访问的服务器端的 IP 地址、要获取的文件路径及文件名、保存到本地的路径和文件名。下载文件时的各相关信息可以保存在控制文件中，若要进行断点续传，只需要给出控制文件名，而各相关的信息可以从控制文件内读取。

（3）客户端连接到指定服务器的约定端口，把要获取文件的路径、文件名、起始偏移量发送给服务器。

（4）服务器打开相应的文件并发送至客户方。若服务器端不能启动文件传输，则需要把原因通行客户方。

（5）客户方接收从服务器端传来的数据并保存到本地磁盘中。在这个过程中，需要同时把与下载有关的信息保存到控制文件中。

（6）双方释放连接。

交互过程可以参考图 3-1。

2、程序设计中的几个要点：

（1）封装文件名的报文格式：

- 一个整型数描述文件名的长度；
- 字符串形式的文件名（含路径）。
- 传输文件的起始偏移量。

（2）服务器方对文件名的响应：

通过向客户发送一个类型为长整型的整数，通过该整数的取值来通知客户方欲获取的文件是否存在，以及文件长度。

- 若能够访问指定的文件，则发送一个长整型的值到客户方，该值为文件的字节长度；
- 若不能访问指定的文件（不存在、无访问权限等），则发送一个小于 0 的整数到客户方。

（3）如何判断指定的文件是否存在？

以只读方式打开指定的文件，若返回的文件描述符大于 0，说明可以访问该文件。

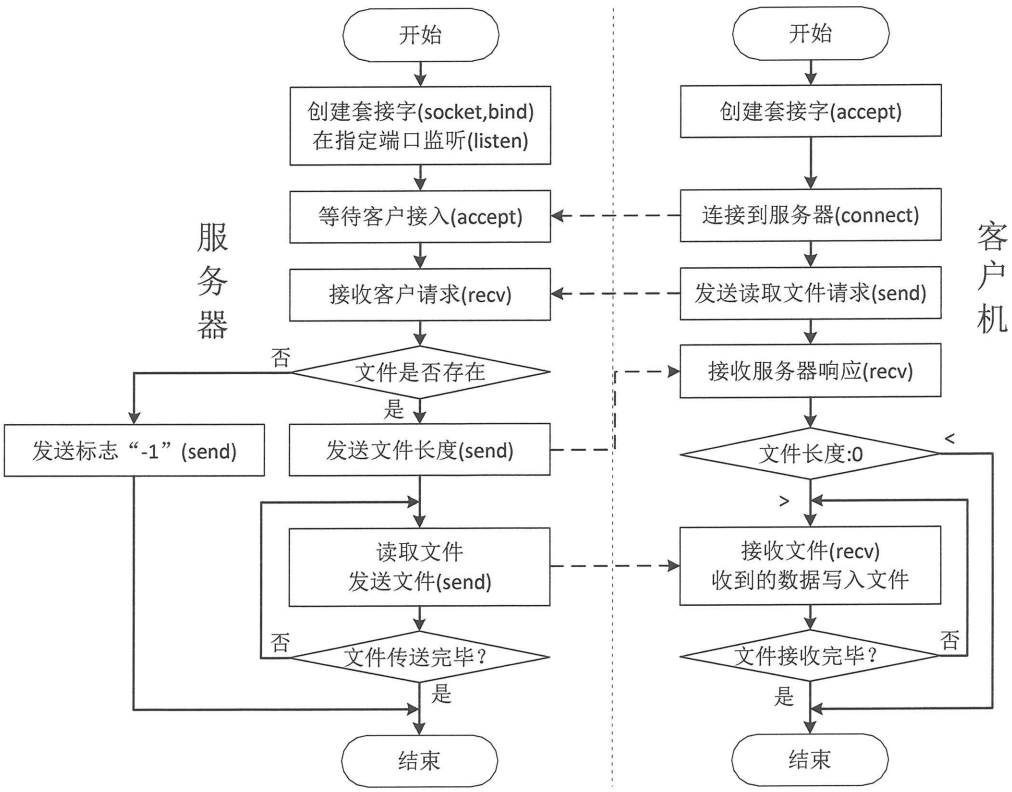


图 3-1 服务器端和客户端程序流程图及相互之间的同步关系

任务二：文件操作指导

在 Linux 环境中对文件的操作有两种方式：

- 系统 I/O：使用系统调用接口函数 open()、read()、write()、lseek()、close()等对文件进行操控，这组函数是由操作系统直接提供的应用程序接口(API)函数。常用于硬件级别，可以设置读缓冲区，一般没有写缓冲区。
- 标准 I/O：C 语言标准库中提供的 I/O 函数 fopen()、fread()、fwrite()、fseek()、fclose()等，这些函数是在对系统 I/O 函数做了进一步封装后来提供。常用于软件级别，自带读写缓冲区。

本实验要求使用系统 I/O 函数对文件进行操作。以下是对使用系统 I/O 函数操作文件进行指导。

对文件的一般操作过程为：打开文件→定位读写指针→读写文件→关闭文件。

(1)打开文件

表 3-1 打开文件调用 open()

功能	打开指定文件或者创建一个新文件，返回文件描述符。		
头文件	sys/types.h		
	sys/stat.h		
	fcntl.h		
原型	int open (const char *pathname, int flags);		
	int open (const char *pathname, int flags, mode_t mode);		
参数	pathname	要打开的文件所在路径及文件名	
	flags	O_RDONLY	以只读方式打开文件
		O_WRONLY	以只写方式打开文件
		O_RDWR	以读写方式打开文件
		O_CREAT	若文件不存在，则创建该文件
		O_TRUNC	若文件已存在，则截断文件内容
		O_APPEND	以追加方式打开文件
	mode	若文件被创建，那么 mode 则指出被创建文件的权限（八进制）	
返回值	成功	大于 0 的整数，作为文件的描述符	
	失败	-1，errno 中含有错误码	

其中 flags 的各种取值可以使用位或运算符连接起来，使得多个标志位共同发挥作用。比如要打开一个文件共读写，若文件不存在则先创建该文件，若文件已存在，则把文件中原有的内容删除掉。那么 flags 参数可以设置为：

```
O_RDWR | O_CREAT | O_TRUNC
```

(2)关闭文件

表 3-2 关闭文件调用 close()

功能	关闭文件并释放相应的资源	
头文件	unistd.h	
原型	int close (fd);	
参数	fd	文件描述符
返回值	成功	0
	失败	-1。错误码在 errno

(3)读文件

表 3-3 读取文件调用 read()

功能	从指定文件中读取数据	
头文件	unistd.h	
原型	ssize_t read (int fd, void *buf, size_t count);	
参数	fd	文件描述符，指定要从哪个文件读取数据。
	buf	存放读取到数据的内存缓冲区首地址。
	count	要读取的字节数。通常设置与缓冲区长度相等。
返回值	成功	实际读到的数据长度。该值小于等于 count。
	失败	-1。错误码在 errno

(4)写文件

表 3-4 写入文件调用 write()

功能	把数据写入到指定的文件。	
头文件	unistd.h	
原型	ssize_t write (int fd, const void *buf, size_t count);	
参数	fd	文件描述符，指定要写入的文件。
	buf	指向要写入文件中的数据所在的缓冲区。
	count	写入的字节数。
返回值	成功	实际写入文件中的字节数。该值小于等于 count。
	失败	-1。错误码在 errno

(5)移动文件指针

文件读写指针指出在使用 read()函数和 write()函数时，从什么位置开始访问文件内容。通过改变文件读写指针的位置，也就设置了从何处开始访问文件。设置文件读写指针位置使用系统调用 lseek()。该系统调用的用法如下：

表 3-5 移动文件读写指针调用 lseek()

功能	设置文件读写指针位置		
头文件	sys/types.h		
	unistd.h		
原型	off_t lseek (int fd, off_t offset, int whence);		
参数	fd	文件描述符。指出要调整读写指针位置的文件。	
	offset	相对基准点(whence)的偏移大小。	
	whence	SEEK_SET	文件开始处。
		SEEK_CUR	文件读写指针的当前位置。
		SEEK_END	文件末尾。
返回值	成功	文件指针当前所在位置相对文件开头的偏移量。	
	失败	-1。错误码在 errno	

(6)访问文件实例

设计一个访问文件的程序，要求具有如下功能：

- 根据用户输入的文件名打开指定文件，并显示该文件的长度。

- 根据用户输入的起始位置，从指定位置开始显示文件内容。

根据上述需求，可以设计出如下程序：

程序清单 3-1 操作文件示例

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <fcntl.h>
void main(void)
{
    int fd;
    off_t offset;
    ssize_t len;
    char fn [20];
    char buf [257];
    printf ("Please input file name: ");
    scanf ("%s", fn);
    if ((fd = open (fn, O_RDONLY)) < 0)
    {
        fprintf (stderr, "Open file error: %s\n", strerror(errno));
        return;
    }
    if ((offset = lseek (fd, 0, SEEK_END)) < 0)
    {
        fprintf (stderr, "Lseek error: %s\n", strerror (errno));
        close (fd);
        return;
    }
    printf ("The length of file \" %s \" is %ld Bytes\n",fn, offset );
    printf ("Please input the read offset: ");
    scanf ("%ld", &offset);
    offset = lseek (fd, offset, SEEK_SET);
    while ((len = read (fd, buf, sizeof(buf) - 1))>0)
    {
        buf[len] = '\0';
        printf ("%s", buf);
    }
    fprintf (stderr, "%s\n", strerror(errno));
    close (fd);
    return;
}
```

实验扩展

请将程序修改为既可以把文件从服务器端下载到客户端，也可以把文件从客户端上传到服务器端。提示：

1、客户端向服务器发送第一个报文时，不但要指出要传输的文件名，还要告诉服务器端软件客户方是想要下载文件还是上传文件；

2、客户端软件需要接收用户指令，使得客户端软件知道用户提出的要求是上传文件还是要下载文件。

补充说明

以下可能是在程序中需要包含的头文件

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
```